Authored by: Arpit Raj, LNMIIT Jaipur
Date: October 23, 2026
© 2026 Arpit Raj. All rights reserved.

OOP: Encapsulation & Access Modifiers

Encapsulation (First Pillar)

- It is the binding of data and methods in a class.
- -> It provides a secure layer.
- -> Hides internal implementation of code & data in a class.
- -> Exposes only necessary info to external world.

Since we are hiding data from the outer class or world, it's also called **data hiding**.

It implements classes in such a way that it:

1. Prevents unauthorized access/modification of contents of class by its objects.
2. The algorithms of one class need not to be known by another class, they can still communicate though.
3. Hides internal implementation of code & data in class, and exposes only necessary info.

Classes & Method Access

Let there be a `Student` class with attributes:

- `name` -> **Public**
- `age` -> **Public**
- `gfname` -> **Private**

And methods:

- `study()` -> **Public**
- `calculate()` -> **Public**
- `love()` -> **Private**

Now:

```
main() {
    Student A("arpit", "21", "aadya");
    cout << A.name;      // -> OK
    cout << A.gfname;    // -> cant access
}
```

Let there be another class `Parent()`.

Then parent can call `A.study()` cause it's **Public**, however even after that it cant know the underlying algorithm of that method.

- Method access -> allowed
- Know external method algorithm -> hidden

`A.love()` is also not allowed as it is **Private**.

Diagram: `name` -> **Public**, `getname()` -> `cout << name`

Access Modifiers

-> Ways to define access to data and methods of a class.

- **Public:** public members are accessible from any part of program.
  - Accessible by objects of class
  - Accessible by external code
- **Private:** private members are:
  - Accessible within same class
  - Not accessible from derived class
  - Not accessible from outside the class
- **Protected:** protected members are:
  - Accessible within same class
  - Accessible by derived classes
  - Not accessible from external code

> 💡 **INFO: Perfect Encapsulation**
>
> If all data members are
> **Private**
> it's called perfect encapsulation.

Getters & Setters

So while `P.name` -> no access
`P.getname()` -> access given (a **Public** function in that class)

(Note: A section of notes here was explicitly crossed out by the author and is omitted)

> ❓ **CONCEPTUAL QUESTION**
>
> **Why use getter setters afterall if a data is private, isnt accessing private data defeating the purpose of making it private?**
>
> **Answer:**
>
> Purpose of
> **getters**
> &
> **setters**
> is to control how the data is accessed and modified.

Eg:

```
private:
    int bal = 1000;

public:
    int getbal() { return bal; }
    void setbal(int amount) {
        if (amount >= 0) {
            bal = amount;
        }
    }
```

Without **getters** and **setters** we cant access or modify the **Private** data, but with help of **getter** and **setter** we have a controlled access to **Private** data.

```
a1.setBal(-1000); // rejected because of regulations of method
a1.setBal(6767);  // allowed
```

Public	Private
Anyone can assign any value	Setter can validate values
No control over modification	Reading & writing can have diff terms

- **getter:** Provides controlled read access.
- **setter:** Provides controlled write access.
- **Private:** Prevent direct access from outside the class.

We dont always need **setters**, **getters** or both as they are prone to bypassing of rules.
Eg `a1.setbal(100000)`
A safer method is:

```
void deposit() {
    if (amount > 0) {
        balance += amount;
    }
}
```

MOST IMPORTANTLY, **getters** and **setters** provide a controlled access to data, we cant directly access the **Private** data, but we can access them via the **Public** methods which can provide some additional checks and constraints.